

4

ENUMERATING OPERATING SYSTEM ARTIFACTS



A normal, “real” user environment looks much different from a malware sandbox or lab environment. A typical user will likely have installed common applications, such as Microsoft Office, email clients, multiple web browsers, and so on. They probably wouldn’t be using a VM, Wireshark, or Procmon, nor are they likely to have installed malware analysis tools such as IDA Pro or sandboxing tools like Cuckoo. A sandbox or lab environment, on the other hand, typically has analysis software installed in a VM.

This is indicated by references to the hypervisor in the names and properties of various *operating system artifacts*, such as the currently running processes, configured registry keys, and installed devices and drivers. Malware can extract this information to understand its environment, choose its target, and decide whether or not to execute its payload on the host. This is known as *enumeration*.

In this chapter, I’ll walk you through enumeration techniques for several OS artifacts and explain how malware might use the information it gathers to identify an analysis environment or inappropriate target. Many of

the techniques we'll explore in this and the next few chapters can be circumvented by a specially tuned malware analysis environment, which we'll discuss in [Appendix A](#).

Processes

Malware can use process enumeration techniques to detect VMs and sandboxes and better understand its operating environment. The primary Windows API functions that malware may use to enumerate processes are `CreateToolhelp32Snapshot`, `Process32First`, and `Process32Next`. The following malware sample excerpt enumerates running processes on a host to look for the target process *VboxTray.exe*, which is common in some VirtualBox VMs:

```
❶ call CreateToolhelp32Snapshot
   --snip--
loc_2:
   lea ecx, [esp+peInfo]
❷ push ecx // Pointer to buffer (peInfo)
   push eax // Handle to snapshot
   call Process32First
❸ test [esp+peInfo.szExeFile], vboxtray_process_name
   jz loc_3
   --snip--
loc_3:
   lea ecx, [esp+peInfo]
❹ push ecx // Pointer to buffer (peInfo)
   push eax // Handle to snapshot
   call Process32Next
   test [esp+peInfo.szExeFile], vboxtray_process_name
❺ jz loc_3 // Loop
```

The `CreateToolhelp32Snapshot` function creates a snapshot of the processes running on the victim's host and returns a handle to it ❶. This handle is pushed to the stack, along with a pointer to the `peInfo` buffer, which will store the output of the `Process32First` function ❷. `Process32First` outputs information about the first process running on the host, `peInfo.szExeFile`. After calling `Process32First`, the malware compares `peInfo.szExeFile` to the `vboxtray_process_name` variable, which was previously defined as

"VboxTray.exe" ❸. (This variable assignment isn't shown in the code snippet.) If the process names don't match, the program jumps to the next area of code.

Both the pointer to the buffer and the process snapshot handle are again pushed to the stack to prepare for the next function: `Process32Next` ❹, which does exactly the same thing as `Process32First` but will iterate over the remaining processes running on the system. After the next process is enumerated and stored in the buffer, the malware does another comparison to the `vboxtray_process_name` variable. This time, if the names don't match, the code will loop to execute `Process32Next` again, repeating until it enumerates all processes or finds the target process ❺.

If you're analyzing malware and you spot it enumerating the process list and checking for a specific process name such as *VboxTray.exe* or *Vmtoolsd.exe*, it should raise a red flag. Here are some of the common VM-related process names malware might be targeting:

VMware

- *TPAutoConnSvc.exe*
- *VGAuthService.exe*
- *VMwareService.exe*
- *Vm3dservice.exe*
- *Vmtoolsd.exe*
- *Vmwaretray.exe*
- *Vmwareuser.exe*

VirtualBox

- *VboxControl.exe*
- *VBoxService.exe*
- *VboxTray.exe*

Some malware variants also search for typical malware analysis tools (such as Procmon, Wireshark, Process Explorer, and Fiddler), which may be running on the analyst's workstation at the time of detonation. If the

malware finds any of the following example processes running on the system, it may decide to terminate itself or take other evasive actions:

- *autoruns.exe*
- *fiddler.exe*
- *ollydbg.exe*
- *procexp.exe*
- *procmon.exe*
- *tcpview.exe*
- *wireshark.exe*
- *x64dbg.exe*

Oftentimes, you can simply rename the executable file for these tools before running the process. For example, renaming *procmon.exe* to *nomcorp.exe* may evade basic malware checks. This trick won't work for malware that looks at the window title bar, however.

When enumerating and searching for specific process names, some malware will use hashed names instead of cleartext strings so that its target won't be readily apparent to the malware analyst. For example, the MD5 hash sum of the process name *fiddler.exe* is 447c259d51c2d1cd320e71e6390b8495, so if you were debugging a particular malware sample or inspecting it in a disassembler, you might notice the string 447c259d51c2d1cd320e71e6390b8495 rather than *fiddler.exe*. You can see how this would slow down analysis and possibly stump an inexperienced analyst.

Hashing in this way will be covered in more depth in [Chapter 16](#). For now, just keep in mind that you might not see nice, cleartext process names when analyzing a malware sample; always be aware of possible hashing. This is true not only for processes but also for filenames, services, and other strings that the malware may be searching for.

Directories and Files

Enumerating directories and files is another technique malware can use to detect sandboxes and VMs. Malware may search for specific files and directories associated with hypervisor software (such as VMware

Workstation and VirtualBox) or with automated malware sandboxes (such as Cuckoo).

The following snippet shows a malware sample calling `FindFirstFile` with `C:\Windows\System32\drivers\vm*` as a target. The sample is searching the Windows *drivers* directory for any driver files with a prefix of *vm*, which is a common pattern for VMware Workstation driver files:

```
call FindFirstFileW
test [ebp+FindFileData.cFileName], fileName ; "C:\Windows\System32\drivers\vm*"
jz   loc_2
--snip--
loc_2:
--snip--
call FindNextFileW
test [ebp+FindFileData.cFileName], fileName
jz   loc_2
```

After the call to `FindFirstFile`, the `test` instruction compares the value of the returned filename to `C:\Windows\System32\drivers\vm*`. If the result of the `test` instruction is 0 (meaning there's no match), the code jumps to `loc_2`, which contains the next function call, `FindNextFile`. `FindNextFile` executes, and the same `test` instruction compares the filenames. If the test instruction returns 0, the program jumps to `loc_2` again. `FindNextFile` will continue to execute until there's a match or there are no more files.

While file enumeration in malware is very common for many reasons, the presence of `FindFirstFile` and `FindNextFile` could mean the malware sample may be attempting to detect an analysis environment. VMware Workstation and VirtualBox file and directory paths that malware might target include the following:

VMware files

- `C:\Windows\System32\drivers\vm*`
- `C:\Windows\System32\vm*`
- `C:\Windows\SysWOW64\vm*`

VMware directories

- *C:\Program Files\VMware*
- *C:\Program Files\Common Files\VMware*

VirtualBox files

- *C:\Windows\System32\VBox**
- *C:\Windows\System32\drivers\VBox**

VirtualBox directories

- *C:\Program Files\Oracle\VirtualBox Guest Additions*

Similar to VMs, automated sandbox environments often contain specific folders and files that malware may find suspicious. Many Cuckoo installations, for example, contain several scripts and files like *Analyzer.py* and *analysis.conf*. Some Cuckoo versions may even have a directory called *cuckoo* or *agent* under the *C:* directory path. Keep this in mind when analyzing evasive malware in an automated malware analysis sandbox.

Finally, malware can use directories and file enumeration to identify its operating environment and profile its target. For example, if a malware sample is targeting a certain organization or type of system, it might enumerate the filesystem to look for specific directories and files containing the target company name. This check would allow the malware to infect only systems in the scope of its attack. A good example of such a system is Stuxnet, mentioned in the introduction. As part of its exploitation tactics, Stuxnet searches for several files related to the Siemens Step7 software on the victim host. If these files don't exist, the system will not be infected.

NOTE

Some of these files and directories can be renamed or even removed without affecting the VM environment. I'll discuss this in more detail in

[Appendix A](#).

Shared Folders

To share files between VMs and the host OS, an analyst can configure *shared folders*. But while shared folders allow easy transfer of files from the host to the guest and vice versa, they also give malware another clue that it's being analyzed.

Malware can enumerate shared folders by using the `WNetGetProviderName` Windows API function, which retrieves network resource information. As VM shared folders are effectively network resources, calling this function on a VirtualBox VM configured with shared folders may return a result of `VirtualBox Shared Folders`, for example, which is a dead giveaway that this machine is hosted on a VirtualBox hypervisor.

Additionally, because VM shared folders function like network drives, they can be identified via normal file and folder enumeration functions, such as the previously discussed `FindFirstFile` and `FindNextFile`. The target path for these functions would be the name of the VM network drive. Some common network drive names are `\\VboxSrv` (for VirtualBox) and `\\vmware-host` (for VMware Workstation).

The Registry

The Windows registry contains information about installed software, hardware configurations, the network, language and location settings, and many other data points that Windows uses to function. As a result, malware attempting to understand the target environment or detect a VM or sandbox will often query the registry. It might enumerate entire hives of the registry and search for suspect VM-related strings, or it might simply inspect a few keys of specific interest. The following excerpt shows how this might look in an actual malware sample:

```
xor    r8d, r8d    ; ulOptions
mov    r9d, ebx    ; samDesired
❶ mov    rdx, r15    ; lpSubKey ("HKLM:\HARDWARE\Description\System")
mov    rcx, r13    ; hKey
mov    [rsp+70h+phkResult], rax ; lpReserved
❷ call  RegOpenKeyExA
```

```
test    eax, eax
jz      short loc_180018C24
--snip--
❸ mov    [rsp+70h+lpData], r14 ; lpData
mov     [rsp+70h+lpType], rax ; lpType
and     [rsp+70h+phkResult], 0
lea     r9, [rbp+cchValueName] ; lpcchValueName
❹ mov     r8, rsi ; lpValueName
mov     edx, r15d ; dwIndex
❺ call   RegEnumValueA
cmp     eax, 0EAh ; 'ê'
jz      short loc_180018D03
--snip--
❻ cmp     [rsp+70h+lpData], suspect_value ; "VBOX -1"
je      terminate_process
```

The malware calls `RegOpenKey` to open a specific key of interest, followed by the `RegEnumValue` function to enumerate all values and data under this registry key. Let's break this code down a bit more.

First, the malware moves the `lpSubKey` value, which represents the registry key it's interested in, into the `rdx` register ❶. In this case, this registry key is `HKLM:\HARDWARE\Description\System`. The value in the `rdx` register is used as a parameter for the subsequent call to `RegOpenKeyExA` ❷. Next, the code loads pointers to the `lpData` ❸ and `lpValueName` ❹ structures. Then, it calls `RegEnumValueA` ❺, which stores the returned values and data for the `HKLM:\HARDWARE\Description\System` registry key in `lpValueName` and `lpData`, respectively.

Typically, malware will call `RegEnumKey` multiple times in a loop, as each iteration of `RegEnumKey` stores only one value and data item. For simplicity's sake, I included only one call to this function in the code.

Finally, the malware compares the suspect value, `vBOX -1`, to the value in the `lpdata` buffer ❻. If they match, the malware assumes it's running in a `VirtualBox` VM and terminates itself.

Malware may also query certain areas of the registry for environment profiling. For example, it might attempt to detect whether it's running on

a computer system of a certain company in order to avoid inadvertently infecting the wrong target. In the following pseudocode, this malware sample is querying the registry for software related to its target, a company called NewCorp:

```
reg_key = RegOpenKey(HKEY_CURRENT_USER, "Software\\NewCorp", result)

if (reg_key != ERROR_SUCCESS) {
    TerminateProcess()
}
```

This example is using the `RegOpenKey` Windows API function to search for the `HKCU\Software\NewCorp` registry key of the host's system. If the malware finds the key, it assumes it's running on a system within the NewCorp organization; otherwise, it terminates itself.

Malware can also query the registry using Windows command line tools, such as `reg query`:

```
C:\> reg query HKLM\Software\NewCorp
```

This command would succeed if the key exists on the victim's system and fail otherwise.

One final way in which malware can query the registry is by using Windows Management Instrumentation (WMI), PowerShell, and other native Windows tools, as we'll discuss in [Chapter 15](#). For now, the point is that there are many methods of querying the registry, but it's also important to note that malware and benign software alike both query the registry for many reasons; a query doesn't always indicate that malware is using a profiling or evasion technique. Registry keys that malware may search for in an attempt to identify a VM analysis environment include the following:

VMware Workstation

- `HKCU:\SOFTWARE\VMware, Inc.`
- `HKLM:\SOFTWARE\Classes\Applications\VMwareHostOpen.exe`

- HKLM:\SOFTWARE\Classes\VMwareHostOpen.AssocFile
- HKLM:\SOFTWARE\Classes\VMwareHostOpen.AssocURL
- HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\VMware User Process
- HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\VMware VM3DService Process
- HKLM:\SOFTWARE\RegisteredApplications\VMware Host Open
- HKLM:\SOFTWARE\WOW6432Node\RegisteredApplications\VMware Host Open
- HKLM:\SYSTEM\CurrentControlSet\Enum\IDE\DiskVMware_Virtual_IDE_Hard_Drive_____00000001\

VirtualBox

- HKLM:\HARDWARE\ACPI\DSDT\VBOX__
- HKLM:\HARDWARE\ACPI\FADT\VBOX__
- HKLM:\HARDWARE\ACPI\RSDT\VBOX__
- HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\VBoxTray
- HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\Oracle VM VirtualBox Guest Additions
- HKLM:\SOFTWARE\Oracle\VirtualBox Guest Additions
- HKLM:\SYSTEM\ControlSet001\services\VBoxMouse
- HKLM:\SYSTEM\ControlSet001\services\VBoxSF
- HKLM:\SYSTEM\ControlSet001\services\VBoxService
- HKLM:\SYSTEM\ControlSet001\services\VBoxVideo
- HKLM:\SYSTEM\ControlSet001\services\VBoxGuest

General

- HKLM:\HARDWARE\Description\System\BIOS\SystemManufacturer
- HKLM:\HARDWARE\Description\System\BIOS\SystemProductName
- HKLM:\HARDWARE\Description\System\SystemBiosVersion
- HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Store\Configuration\OEMID
- HKLM:\SYSTEM\CurrentControlSet\Control\Class\

You can use the Windows tool *regedit.exe* on your own analysis VM to investigate these keys and get an idea of what malware may be looking for. For example, when I inspect the HKLM:\HARDWARE\Description\System registry key in my own analysis machine, I can spot certain values and data that may be interesting to malware (see [Figure 4-1](#)).

Name	Type	Data
(Default)	REG_SZ	(value not set)
BootArchitecture	REG_DWORD	0x00000003 (3)
Capabilities	REG_DWORD	0x00000541 (1345)
Component Information	REG_BINARY	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Configuration Data	REG_FULL_RESOU...	ff ff ff ff ff ff ff ff ff ff 00 00 00 02 00 00 00 05 00 00 00 0c 00 00 00 00 00 00 00 00 00
Identifier	REG_SZ	AT/AT COMPATIBLE
PreferredProfile	REG_DWORD	0x00000000 (0)
SystemBiosDate	REG_SZ	06/23/99
SystemBiosVersion	REG_MULTI_SZ	VBOX -1
VideoBiosVersion	REG_MULTI_SZ	Oracle VM VirtualBox Version 6.1.15 VGA BIOS Oracle VM VirtualBox Version 6.1.15

Figure 4-1: The hardware description in regedit.exe

The obvious data points of interest here are the `VBOX -1` and `Oracle VM VirtualBox` strings. Another big hint is the `SystemBiosDate` of `06/23/99`, which indicates the VirtualBox hypervisor. Once again, these registry keys are only examples. They may or may not exist on your VMs, depending on your hypervisor version, installed software, and other factors.

NOTE

These registry keys can typically be renamed or removed completely in order to trick malware and bypass its evasion techniques. I'll discuss this in more detail in [Appendix A](#).

Services

Services are applications that typically start at system boot-up and run in the background of a user's session, and they're yet another way in which malware may attempt to identify a VM or sandbox environment. Some services are unique to a certain brand of hypervisor, giving away its presence, and some services can also identify installed malware analysis tools. For example, malware may be looking for the *VirtualBox Guest Additions Service*, which indicates the presence of VirtualBox.

Malware may query services in multiple ways. One of the most common techniques is using the Windows API function `EnumServiceStatus` or `EnumServiceStatusEx`, which will return a list of services on the host and their associated statuses, which are usually either `running` or `stopped`. Malware can also use the function `OpenService` to open a handle to a specific service. If it successfully obtains a handle (meaning the service exists), the return code will be `true`.

In addition to Windows API functions, there are other methods of enumerating services that malware can employ. For example, malware can use the Windows `sc` command line tool to query a list of services or a specific service, like so:

```
C:\> sc query ServiceName
```

Similarly, malware can use the WMI command line tool `wmic` to query all services on the host:

```
C:\> wmic service get name
```

Finally, since some service information is stored in the registry, malware can query the registry for service information. If you see malware using service-related API functions, using tools such as `sc`, or querying the registry to find specific hypervisor services, the sample may be attempting to detect an analysis environment.

To better understand and explore the services running on your VMs and sandboxes, you can use the Windows native application `services.exe` to get a list of services configured on the system. In [Figure 4-2](#), you can see the VirtualBox Guest Additions Service running, which can signal to malware that it is running in a VM environment.

Name	Description	Status
Windows Audio Endpoint Builder	Manages audio devices for the Windows Audio service. ...	Started
Windows Audio	Manages audio for Windows-based programs. If this se...	Started
WebClient	Enables Windows-based programs to create, access, an...	
Volume Shadow Copy	Manages and implements Volume Shadow Copies used...	
VirtualBox Guest Additions Service	Manages VM runtime information, time synchronizatio...	Started
Virtual Disk	Provides management services for disks, volumes, file s...	
User Profile Service	This service is responsible for loading and unloading us...	Started
UPnP Device Host	Allows UPnP devices to be hosted on this computer. If t...	Started
TPM Base Services	Enables access to the Trusted Platform Module (TPM), ...	

Figure 4-2: The output of `services.exe`

[Table 4-1](#) lists some of the common services created by VMware and VirtualBox.

Table 4-1: Common Hypervisor Services

Name	Description
VGAuthService	VMware Guest Authentication Service
VMTools	VMware Tools Service
VBoxService	VirtualBox Guest Additions Service

Installed Software

Malware may enumerate the installed software on your analysis machine and look for keywords that represent common analysis tools or sandbox-related artifacts. This technique can also be used for target profiling; malware may simply query the host, looking for installed software indicating that this system is a valid target.

To search for installed software, the malware might use a Windows API function such as `MsiEnumProducts` or inspect the `HKLM\Software\Microsoft\Windows\Current Version\Uninstall` registry key, whose contents from my malware analysis machine are shown in **Figure 4-3**.

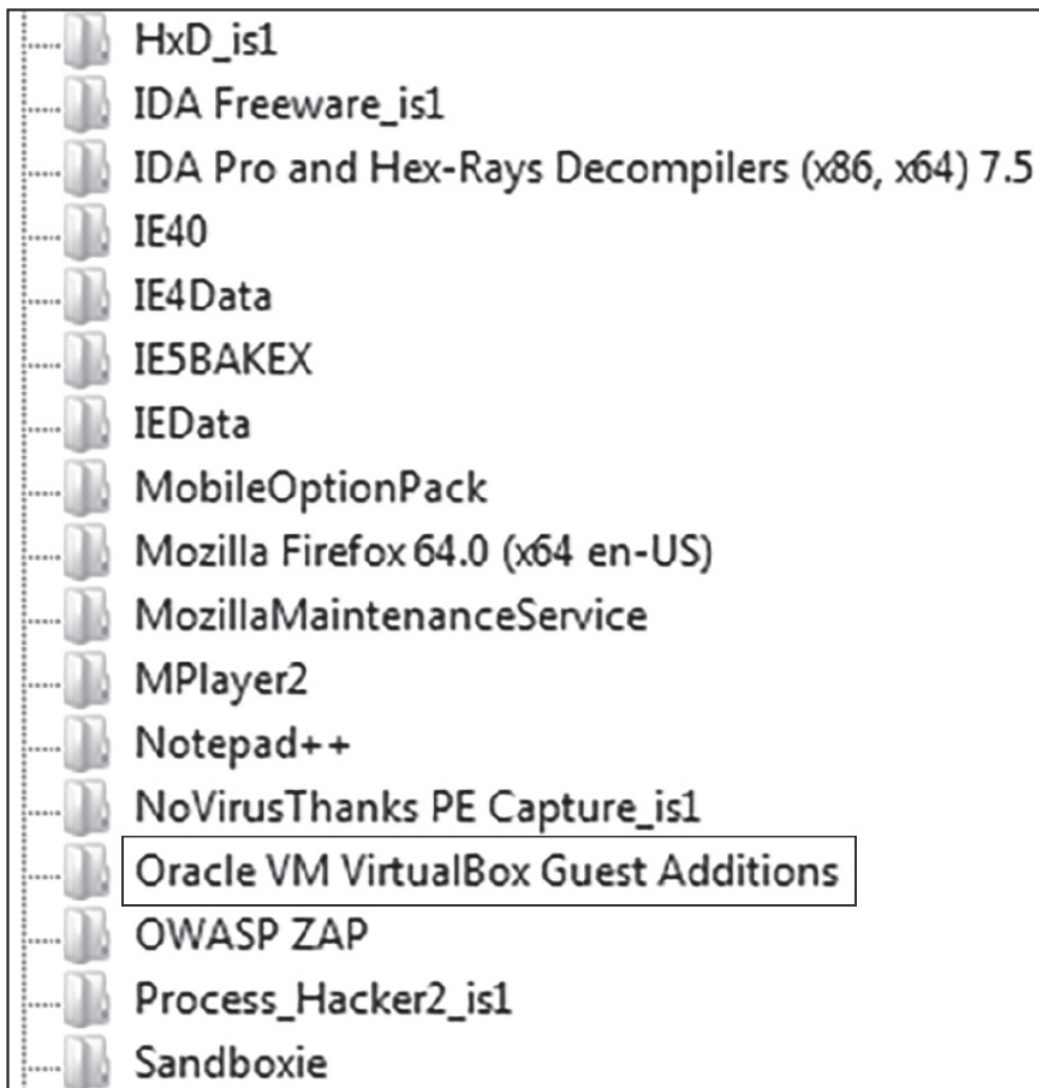


Figure 4-3: Enumerating the HKLM\Software\Microsoft\Windows\Current Version\Uninstall registry key

Here you can see several analysis tools, as well as VirtualBox Guest Additions.

NOTE

Software installers create many artifacts on the host, such as new folders, files, and registry entries, all of which can be enumerated by malware. Sometimes it's a better approach to use the portable versions of analysis tools, which don't need to be installed and are run directly from a self-contained executable file.

Mutexes

As [Chapter 1](#) explained, a *mutex* is an object that helps control access to resources, acting as a sort of gatekeeper. Certain sandboxes and hypervi-

sors have unique mutexes that may be interesting to malware. For example, VMware typically uses mutexes that begin with *VMware*, such as `VMwareGuestCopyPasteMutex`.

Malware can use the Windows API function `CreateMutex` OR `OpenMutex` to search for specific mutex values, as shown in this sample:

```
push mutex_name ; "VMwareGuestCopyPasteMutex"
push 0
push 20000L
call OpenMutexA
test eax, eax
```

The malware pushes the `mutex_name` variable, which contains the mutex that the malware wants to find (`VMwareGuestCopyPasteMutex`), to the stack. It also pushes two other values that are required for the `OpenMutex` function but aren't important here. Next, it calls `OpenMutexA` and uses a `test` instruction to check whether that function succeeded or failed. If the function call succeeded, `VMwareGuestCopyPasteMutex` exists on the system, meaning that the system is likely running inside a VMware hypervisor.

Keep in mind that not all mutex-related activity is malicious. Mutexes are standard Windows objects and are not inherently malicious *or* benign. If you spot malware enumerating mutexes and attempting to find very specific strings such as the examples just given, it could be using this detection technique. Some mutexes that may reveal a VM environment include the following:

VMware Workstation

- `VMToolsHookQueueLock`
- `VMwareGuestCopyPasteMutex`
- `VMwareGuestDnDDDataMutex`

VirtualBox

- `VBoxService`
- `VBoxTray`

Pipes

A *pipe* is a Windows object that allows for interprocess communication. Some hypervisors create unique pipes on the guest OS that may give away their presence to malware. Malware can call several different functions, such as `CreateFile` and `CallNamedPipe`, to search for specific named pipes.

A great way to check your own VMs for these pipes is to use the tool `Pipelist`, which is part of the Windows Sysinternals suite from Mark Russinovich. **Figure 4-4** shows the output from running the `pipelist` command on a VMware Workstation VM. If you look closely, you may spot the `vgauth-service` pipe, which is unique to VMware.

```
PipeList v1.02 - Lists open named pipes
Copyright (C) 2005-2016 Mark Russinovich
Sysinternals - www.sysinternals.com
```

Pipe Name	Instances	Max Instances
-----	-----	-----
InitShutdown	3	-1
lsass	4	-1
ntsvcs	3	-1
scerpc	3	-1
Winsock2\CatalogChangeListener-2d4-0	1	1
Winsock2\CatalogChangeListener-3c4-0	1	1
epmapper	3	-1
Winsock2\CatalogChangeListener-22c-0	1	1
LSM_API_service	3	-1
Winsock2\CatalogChangeListener-3f8-0	1	1
eventlog	3	-1
Winsock2\CatalogChangeListener-4d4-0	1	1
atsvc	3	-1
Winsock2\CatalogChangeListener-50c-0	1	1
spoolss	3	-1
Winsock2\CatalogChangeListener-b04-0	1	1
wkssvc	4	-1
trkwks	3	-1
Winsock2\CatalogChangeListener-2bc-0	1	1
srvsvc	4	-1
vgauth-service	1	-1
ROUTER	3	-1
MsFteWds	4	-1

Figure 4-4: Output from running the `pipelist` command on a VMware Workstation VM

Here are some of the common pipes that may reside on your VMs:

VMware Workstation

- `Vmmemctl`
- `vgauth-service`

VirtualBox

- VBoxTray
- VBoxTrayIPC
- VBoxGuest
- VBoxVideo
- VBoxMouse
- VBoxMiniRdr

Devices and Drivers

Hypervisors often install specific devices and drivers on guest operating systems. *Devices* are objects that often represent physical hardware on the system, such as USB controllers or hard disk volumes. *Drivers* are software objects that control hardware, allowing the OS and devices to communicate. Device and driver objects are typically managed by the Windows object manager.

Malware can enumerate devices and driver objects using two Windows functions: `NtOpenDirectoryObject` and `NtQueryDirectoryObject`. The following code shows these functions in action:

```
mov    [rbp+57h+ObjectAttributes.ObjectName], rax
mov    edx, 1      ; DesiredAccess
❶ lea   rcx, [rbp+57h+DirectoryHandle] ; "\\Driver"
❷ call  NtOpenDirectoryObject
mov    ebx, eax
test   eax, eax
--snip--
mov    rcx, [rbp+57h+DirectoryHandle] ; "\\Driver"
xor    r9d, r9d    ; ReturnSingleEntry
mov    [rsp+110h+ReturnLength], rax ; ReturnLength
mov    r8d, r14d   ; BufferLength
mov    [rbp+57h+var_BC], eax
❸ mov   rdx, rsi   ; Buffer
lea    rax, [rbp+57h+var_BC]
mov    [rsp+110h+Context], rax ; Context
mov    [rsp+110h+RestartScan], 1 ; RestartScan
❹ call  NtQueryDirectoryObject
```

You can see a handle to the `\\Driver` directory being loaded into register `rcx` ❶, using the instruction `lea`, or *load effective address*. The `lea` instruction calculates the address of an operand and loads it into a target register. This is followed by a call to `NtOpenDirectoryObject`, which opens `\\Driver` in preparation for the subsequent `NtQueryDirectoryObject` function call ❷. Then, `NtQueryDirectoryObject` ❸ queries the `\\Driver` directory objects and stores the list of objects in a buffer specified previously ❹. The malware then searches through this buffer, looking for specific drivers often used in VMs. Devices can be queried in the same way.

To better understand the contents of the Windows object manager, you can view it directly on your Windows VMs using the WinObj tool from Sysinternals. **Figure 4-5** shows some of the VirtualBox-related drivers installed on my analysis VM using WinObj. Feel free to explore them on your own analysis system.

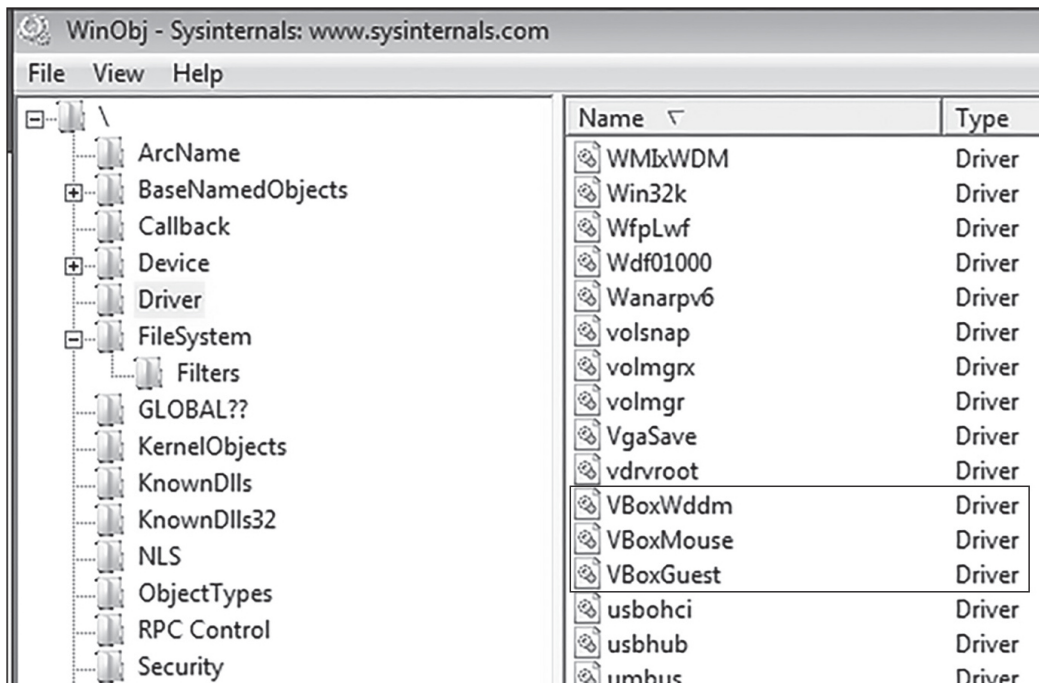


Figure 4-5: Exploring drivers with WinObj

Some common hypervisor drivers for VMware Workstation and VirtualBox include the following:

VMware Workstation

- `vm3dmp`

- vm3dmp-debug
- vm3dmp-stats
- vm3dmp-sloader
- vmci
- vmhgfs
- VMemCtl
- vmmouse
- vmrawdsk
- vmusbmouse
- vsock

VirtualBox

- VBoxGuest
- VBoxMouse
- VBoxSF
- VBoxVideo
- VBoxWddm

And here are some common devices:

VMware Workstation

- VMCIGuestDev
- VMCIMHostDev
- vm3dmpDevice
- vmci
- vmmemctl

VirtualBox

- VBoxGuest
- VBoxMiniRdr

Username and Hostnames

Many automated malware analysis sandboxes have a default list of possible usernames assigned to user accounts on the system. These usernames

may be randomized, but they're often hardcoded. System hostnames may similarly be hardcoded and randomized at boot-up time. This isn't simply a characteristic of automated sandboxes, however. Malware analysts may also configure their analysis VMs with usernames or hostnames that may be generic, well known, or otherwise not how a "real" user would configure their system.

Some malware can take advantage by enumerating the user accounts and hostname of the system, specifically searching for generic usernames such as *Administrator*, *User*, or *John*, or hostnames such as *Cuckoo*, *Test*, *Desktop*, *Workstation*, or *Lab*. *TequilaBoomBoom*, at one point the hostname for VirusTotal's sandboxing service, is also a common check.

NOTE

Configure your VMs and sandbox environments with nongeneric usernames and hostnames. Try to use values that an actual end user or enterprise environment would use, or randomize the names.

Locale and Language Settings

Keyboard and language settings can be used to determine a potential victim's location. Malware might use this technique to include or exclude a host as a valid target for geographic or geopolitical reasons, or to evade analysis. For example, say a malware sample wants to determine whether a potential victim is located in Russia or speaks Russian. Depending on this information, it may then choose to deploy its payload on the host or quietly terminate itself. Here are three different approaches it can use to extract this information:

- Getting the keyboard layout language
- Enumerating the language settings of the system itself, such as the display language
- Obtaining the host's locale

The first method is perhaps the most common. The Windows function `GetKeyboardLayout` returns the active keyboard language of the host, and

`GetKeyboardLayoutList` returns a complete list of all keyboard languages that are installed on the host.

There are several possible Windows functions to get the host's language settings. `GetUserDefaultUILanguage`, for example, returns the currently logged-in user's user interface language. `GetSystemDefaultUILanguage` returns the system language, or more specifically, the language in which the OS was installed. Finally, `GetProcessPreferredUILanguages` will list languages that the user's running processes may be utilizing. Talk about options!

The *locale* of the host is different from the host's keyboard and UI language settings in that it's a list of language-related settings for a particular language-and-country combination. Some items that may be included as part of a locale are currency and date/time display formats, as well as a language identifier. The Windows API functions `GetSystemDefaultLCID` and `GetUserDefaultLCID` return locale information that can be used to profile a host or user. Processes and threads may also have their own custom locales, which malware can query using the function `GetThreadLocale`.

In a malware sample, you'd likely see a call to one of the previously mentioned functions, such as `GetKeyboardLayout`, followed by a comparison to several values that represent Windows language identifiers. The following pseudocode shows this technique in action:

```
keyboard_layout = GetKeyboardLayout(0)
if keyboard_layout == "0x419" {
    TerminateProcess()
}
```

This malware sample calls the `GetKeyboardLayout` function and compares the result to the value `0x419`, which is the Russian language identifier in hexadecimal (also known as *in hex*). If this Russian language identifier is in use, the malware executes `TerminateProcess`.

Table 4-2 lists some of the language identifiers for Windows.

Table 4-2: Common Language Identifiers

Identifier (in hex)	Language (and associated country)
0x402	Bulgarian (Bulgaria)
0x4	Chinese (Simplified) (China)
0x809	English (United Kingdom)
0x409	English (United States)
0x407	German (Germany)
0x418	Romanian (Romania)
0x419	Russian (Russia)

There are several other, more covert methods for enumerating or inferring a potential victim's language and locale settings, such as enumerating the date/time formats, currency formats, or even calendar information. The potential list is so extensive that a portion of this book could be dedicated to this technique alone. What's most important is that you now have the necessary knowledge to identify many of the common methods that malware may use to profile a victim's language and locale.

Operating System Version Information

Before infecting a victim machine, malware often needs to determine whether that machine is running a certain OS. Stuxnet, for example, infected only Windows XP computers because its authors knew that the target facilities used Windows XP to control the Siemens programmable logic controllers. A malware sample that can run on only a certain version of Windows may attempt to identify the OS before infecting the host to avoid inadvertently crashing it. For example, if the malware contains exploit code that works for only a specific sub-version of Windows, such

as Windows 7 Service Pack 1, but causes system instability for Windows 7 Service Pack 2, it might want to determine a victim's OS sub-version first to prevent an unintended crash that would potentially alert the victim to its presence.

This technique isn't usually implemented as a detection or evasion technique, but it's worth discussing in this context because it can still interfere with the analysis process. For example, if you were researching Stuxnet and decided to run the sample in a VM or sandbox with a modern Windows OS, it likely wouldn't run correctly, thereby unintentionally evading automated and dynamic analysis methods. This is an important point to keep in mind: a malware sample that doesn't execute correctly in your analysis environment isn't necessarily intentionally trying to be evasive.

There are multiple ways of enumerating the target's OS and version via the Windows API. The `GetVersionEx` function, along with its close cousin `GetVersion`, accomplishes this, as shown here:

```
lea    eax, [ebp-0A0h]
push   eax
mov     [ebp+VersionInformation.dwOSVersionInfoSize], 9Ch
call    GetVersionExA
test    eax, eax
jnz     short loc_2
--snip--
cmp     [ebp+VersionInformation.dwMajorVersion], 4
jnz     loc_1000FA0F
```

The `GetVersionExA` function takes a buffer as an argument. This buffer will be the area of memory where the returned OS information will be stored after `GetVersionExA` is called. The `lea` instruction loads the buffer address, `[ebp-0A0h]`, into the `eax` register, and then the buffer address is pushed to the stack with `push eax`.

Next, the sample must define the size of the data that will be returned from the `GetVersionExA` function call. This data will be stored in a structure

called `VersionInformation`. This sample specifies the size of the `VersionInformation` structure as `9c`, which in hex is equal to 156 bytes.

Finally, the malware sample calls `GetVersionExA` (call `GetVersionExA`) to get the OS version, then later checks that information by comparing the `dwMajorVersion` with the value `4`, which represents a very old version of Windows. (For some perspective, `5` is the version number for Windows XP!) Essentially, this malware sample is testing to see how old this host's Windows version is.

Table 4-3 contains a subset of the `dwMajor` and `dwMinor` OS versions.

Table 4-3: `dwMajor` and `dwMinor` OS Versions

Operating system	<code>dwMajorVersion</code>	<code>dwMinorVersion</code>
Windows 10	10	0
Windows Server 2016	10	0
Windows 8.1	6	3
Windows Server 2012	6	2
Windows 7	6	1
Windows Server 2008	6	0
Windows Vista	6	0
Windows XP	5	1

Summary

In this chapter, we covered many common (and some not-so-common) ways in which malware can detect an analysis environment by inspecting

operating system objects and artifacts. In the next chapter, we'll explore some techniques malware can use to look for evidence of legitimate user activity (or lack thereof!) to reveal the underlying VM or sandbox environment.